

A Simple Explanation of the Softmax Function

What Softmax is, how it's used, and how to implement it in Python.

JULY 22, 2019 | UPDATED DECEMBER 26, 2019

Softmax turns arbitrary real values into probabilities, which are often useful in Machine Learning. The math behind it is pretty simple: given some numbers,

1. Raise [e](#) (the mathematical constant) to the power of each of those numbers.
2. Sum up all the exponentials (powers of e). This result is the *denominator*.
3. Use each number's exponential as its *numerator*.
4. $\text{Probability} = \frac{\text{Numerator}}{\text{Denominator}}$.

Written more fancily, Softmax performs the following transform on n numbers $x_1 \dots x_n$:

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

The outputs of the Softmax transform are always in the range $[0, 1]$ and add up to 1. Hence, they form a **probability distribution**.

A Simple Example

Say we have the numbers -1, 0, 3, and 5. First, we calculate the denominator:

$$\begin{aligned}\text{Denominator} &= e^{-1} + e^0 + e^3 + e^5 \\ &= \boxed{169.87}\end{aligned}$$

Then, we can calculate the numerators and probabilities:

x	Numerator (e^x)	Probability ($\frac{e^x}{169.87}$)
-1	0.368	0.002



x	Numerator (e^x)	Probability ($\frac{e^x}{169.87}$)
3	20.09	0.118
5	148.41	0.874

The bigger the x , the higher its probability. Also, notice that the probabilities all add up to 1, as mentioned before.

Implementing Softmax in Python

Using [numpy](#) makes this super easy:

```
import numpy as np

def softmax(xs):
    return np.exp(xs) / sum(np.exp(xs))

xs = np.array([-1, 0, 3, 5])
print(softmax(xs)) # [0.0021657, 0.00588697, 0.11824302, 0.87370431]
```

[np.exp\(\)](#) raises e to the power of each element in the input array.

Note: for more advanced users, you'll probably want to implement this using the [LogSumExp](#) trick to avoid underflow/overflow problems.

Why is Softmax useful?

Imagine building a [Neural Network](#) to answer the question: *Is this picture of a dog or a cat?*

A common design for this neural network would have it output 2 real numbers, one representing *dog* and the other *cat*, and apply Softmax on these values. For example, let's say the network outputs $[-1, 2]$:

Animal	x	e^x	Probability
Dog	-1	0.368	0.047
Cat	2	7.39	0.953

This means our network is **95.3% confident** that the picture is of a cat. Softmax lets us



simpler answers (e.g. binary yes/no).

I write about [ML](#), [Web Dev](#), and [more topics](#). **Subscribe to get new posts by email!**

example@domain.com

☐ Send me *only* ML posts

SUBMIT

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

This blog is [open-source on Github](#).

Tags:

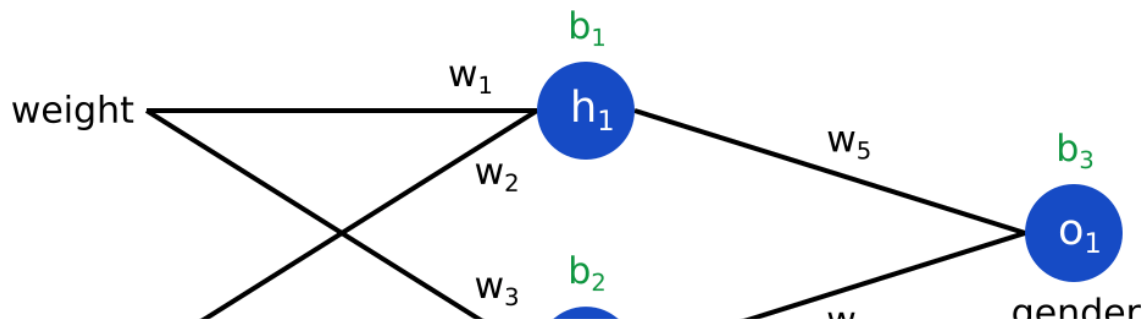
Machine Learning

For Beginners

Neural Networks

Python

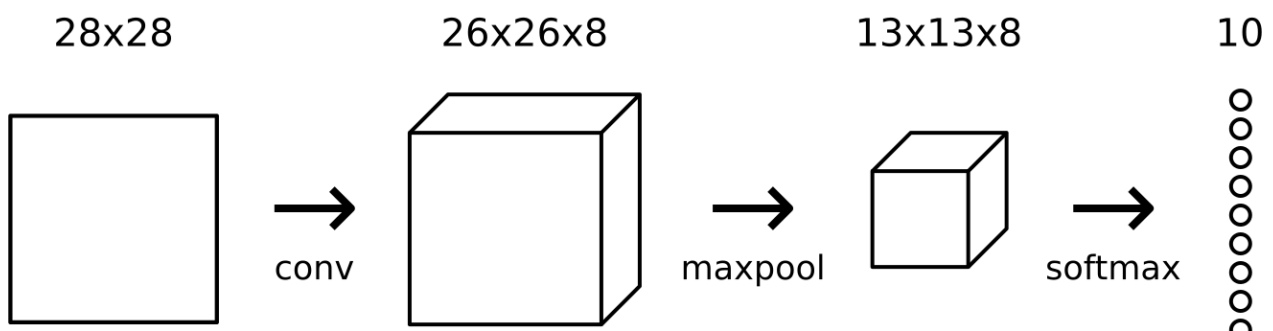
YOU MIGHT ALSO LIKE



Machine Learning for Beginners: An Introduction to Neural Networks

July 24, 2019

A simple explanation of how they work and how to implement one from scratch in Python.



CNNs, Part 1: An Introduction to Convolutional Neural Networks

November 10, 2019

A simple guide to what CNNs are, how they work, and how to build one from scratch in Python.



Victor Zhou [@victorc Zhou](#)

Software Engineer. I blog about [web development](#), [machine learning](#), and [more topics](#).



Victor Zhou

[Blog](#)

[About](#)

[Tags](#)

[More Posts](#)



Add a comment

M ↓ MARKDOWN

☐ COMMENT ANONYMOUSLY

ADD COMMENT

Upvotes Newest Oldest

**Dave Kinney****1 point** · 5 months ago

FINALLY, someone explains softmax in a way that makes sense and I can understand. This makes me very happy, thank you Victor.

J

Jacek Pliszka**1 point** · 20 months ago

You should not write 'network is 95.3% confident that the picture is of a cat'. This is true in some very specific cases but normally is not. The fact that softmax gives us a probability distribution does not mean that it is relevant to our case. It is just some estimator - you may get 95.3% from it but the actual probability that the picture is of a cat may be completely different. Just consider the case when your network returns 1 and 2 after sum over some area, then you sum over 4x larger area and your numbers are 4 and 8 - suddenly your confidence increases from 73% to 98%

A

alternate reality**0 points** · 2 months ago

I agree with what you said. It's not correct to call them probability / confidence score. It is a form of normalising arbitrary values using exponential distribution.

Ran this to verify your argument. You are right to point that scaling the numbers shouldn't affect the output, but it did.

```
import matplotlib.pyplot as plt
import numpy as np

def softmax(xs): return np.exp(xs) / sum(np.exp(xs))

x1 = [1,1.5,2,2.5,3] x2 = np.multiply(x1,4)

plt.scatter(x1,softmax(x1))
plt.plot(x1,softmax(x1))
plt.scatter(x2,softmax(x2))
```

**Victor Zhou**[Blog](#) [About](#) [Tags](#) [More Posts](#)



Apurva Singh

0 points · 11 months ago

Good thx

S

Shlomi Tsur

0 points · 21 months ago

thanks you

?

Anonymous

0 points · 21 months ago

Hi Victor, I am brand new to ML, AI and so on and maybe is a silly question, but doesn't do the same also the sigmoid function (when the output is between 0 and 1)? If so, I really did not catch the essence to choose the right function (both functions have derivatives first and second and partial can be built in easily). Thank you for your advice, Angelo

?

Anonymous

0 points · 12 months ago

Softmax is applicable for multinomial cases whereas sigmoid is what you want for binary classification.

?

Anonymous

0 points · 2 months ago

what i think is that there is one difference in softmax and simoid which is the sum of the outputs is not necessarily 1 in the latter, but is 1 in softmax.

S

Suleymanzade

0 points · 20 months ago

I think that full answer then must be `soft_max = np.maximum(np.exp(x) / sum(np.exp(x)))` or you must return the whole set of distributed numbers??? Thanks

V

Victor Zhou MODERATOR

0 points · 20 months ago

Softmax is just the transform itself - you can take the max over the result of Softmax to find the predicted result, but that's separate.



Rustam

0 points · 20 months ago

Simple and clear. Thank you. Any comments on e function choice? Why not squaring? It will give the same result: values are from 0-1, and sum to 1. Even abs works the same.

?

Anonymous

0 points · 19 months ago

$\frac{1}{2}$



Victor Zhou

[Blog](#)

[About](#)

[Tags](#)

[More . . .](#)



Its a sampled probability but this does not require the use of $\text{Exp}(x)$ The meaning of negative values in the input also raise a question so let's assume the bias is chosen so that we don't have negative values.

Saw some comment saying its not a probability because if you scale the sample area the values changes. A probability is by no means in dependant for the data you choose to look at. Its not a unique value.

For those who ask why using $\text{Exp}(x)$ i think i recall because it has interesting properties for the derivative which make many calculations convenient. $\text{Abs}(x)$ is not a good choice because derivative is discontinuous near 0.



Anonymous

0 points · 10 months ago

Thank you very much!

Powered by **Commento**